

INFORME FINAL

MANUAL DE ARQUITECTURA Y DESARROLLO DE FRAMEWORK MODULAR EN ESP32 V1.0

LOTE V

LOS ORGANOS, NOVIEMBRE 2025.

CONTENIDO

1. INTRODUCCION	3
1.1. PROPOSITO DEL FRAMEWORK	3
1.2. FILOSOFIA Y PRINCIPIO DE DISEÑO	3
1.3. ALCANCE DEL DOCUMENTO	4
1.4. ESTRUCTURA DE PROYECTO	4
2. CONFIGURACION DEL ENTORNO DE DESARROLLO	6
2.1. REQUISITOS PREVIOS	6
2.2. INSTALACION DE DRIVER	6
2.3. INSTALACION DE FIRMWARE DE MICROPYTHON	7
2.4. DESPLIEGUE DEL FRAMEWORK	9
3. ARQUITECTURA DEL SISTEMA	9
3.1. MODELO DE CAPAS (LAYERED ARCHITECTURE)	9
3.2. FLUJO DE DATOS: LENTO VS. REACTIVO	11
4. CAPA DE HARDWARE Y DRIVERS (HAL)	12
4.1. NIVELES DE ENTIDADES DE HARDWARE	12
4.2. ESTRATEGIA DE INTERRUPCIONES EN DOS ETAPAS (TWO-STAGE IRQ)	13
4.3. CICLO DE VIDA DEL HARDWARE	14
5. SISTEMA DE MODULOS (CAPA DE APLICACION)	15
5.1. LA CLASE _BASEMODULE	15
5.2. EL REGISTRO DE MÓDULOS (MODULE_REGISTRY)	16
5.3. CICLO DE EJECUCIÓN DEL MÓDULO	17
6. PROTOCOLO Y RED (CAPA DE COMUNICACION)	18
6.1. DEFINICIÓN DE PAQUETES (PROTOCOL.PY)	18
6.2. PIPELINE DE PROCESAMIENTO DE MENSAJES	18
7. GUIA DE DESARROLLO	20
8. CONCLUSIONES	23
ANEXO A	24
ANEXO B	24
ANEXO C	25

1. INTRODUCCION

1.1. PROPOSITO DEL FRAMEWORK

El desarrollo de aplicaciones embebidas en MicroPython para plataformas como el ESP32 enfrenta un desafío recurrente: la complejidad. Gestionar hardware heterogéneo, coordinar tareas simultáneas, manejar estados y permitir la reconfiguración en tiempo de ejecución a menudo conduce a proyectos monolíticos, difíciles de escalar y con bajo potencial de reutilización.

El Framework Modular para ESP32 nace como una solución directa a este problema. Su misión es proporcionar una arquitectura de software robusta, extensible y mantenible que sirva como una "capa de ingeniería" sobre MicroPython.

El objetivo es claro: **permitir que el desarrollador se enfoque en la lógica de la aplicación (el "qué"), mientras el framework se encarga de la infraestructura (el "cómo")**. Esto se traduce en beneficios directos:

- Acelera el desarrollo de nuevos proyectos al proporcionar una base probada.
- Garantiza una estructura modular y un código limpio y legible.
- Simplifica la integración de hardware diverso a través de una capa de abstracción.
- Habilita comportamientos dinámicos, como cambios de configuración en caliente.
- Aumenta la confiabilidad del sistema al reducir el código repetitivo y propenso a errores.

1.2. FILOSOFIA Y PRINCIPIO DE DISEÑO

El framework está construido sobre cuatro pilares fundamentales:

1. Arquitectura Dirigida por Configuración (Configuration-Driven)

En lugar de codificar el comportamiento en la lógica, este se *declara* en archivos de configuración. El hardware que se usará, los módulos que se ejecutarán y sus parámetros, permitiendo adaptar un proyecto completo a nuevas necesidades sin modificar el código fuente.

2. Capa de Abstracción de Hardware (HAL - Hardware Abstraction Layer)

El hardware se describe mediante un modelo unificado. Los módulos de software interactúan con los dispositivos por su nombre lógico en lugar de hacerlo por su pin físico o dirección I2C. Esto desacopla completamente el software del hardware, mejorando la portabilidad y facilitando las pruebas.

3. Sistema Modular Basado en Máquinas de Estados

Cada funcionalidad se encapsula en un módulo independiente con su propio ciclo de vida, temporizadores y estados internos. Esta arquitectura produce aplicaciones organizadas, escalables y fáciles de depurar, donde cada componente tiene una responsabilidad única y bien definida.

4. Gestión de Estado Dual: Memoria vs. Reflejos

El framework implementa un modelo híbrido de gestión de información inspirado en sistemas biológicos:

- **Pizarra Global (board.py):** Actúa como la "memoria" del sistema. Hardware y Drivers escriben sus estados aquí (nivel bajo), y los Módulos leen o escriben datos (nivel alto). Es ideal para información que no requiere reacción inmediata.
- **Sistema Reactivo (pubsub.py):** Actúa como el "sistema nervioso". Se utiliza para eventos críticos que requieren una respuesta instantánea, evitando el tiempo de latencia de lectura normal.

5. Modelo de Concurrency Determinista (Polling vs. Asyncio)

A diferencia de soluciones basadas en asyncio, este framework utiliza un bucle principal basado en Timer no bloqueante y polling de alta frecuencia. Esta decisión de diseño permite realizar lecturas intensivas de hardware sin que el "await" de una tarea bloquee a las demás. Esto garantiza que el sistema pueda atender periféricos rápidos mientras mantiene vivos los ciclos de vida de los módulos de alto nivel.

1.3. ALCANCE DEL DOCUMENTO

Este manual es una guía completa de la arquitectura, los conceptos y los mecanismos internos del framework. Su contenido incluye:

- Una guía de inicio rápido para configurar el entorno y ejecutar un primer proyecto.
- Una descripción detallada de cada componente: Gestor de Configuración, HAL, Sistema de Módulos y Motor de Eventos.
- Buenas prácticas de diseño, ejemplos de uso y patrones de desarrollo recomendados.

El objetivo es que el lector comprenda no solo cómo usar la plataforma, sino también cómo está construida y por qué se tomaron ciertas decisiones de diseño.

1.4. ESTRUCTURA DE PROYECTO

El framework organiza el código en una estructura de carpetas y archivos diseñada para maximizar la claridad y la separación de responsabilidades. La disposición general de un proyecto que utiliza este framework es la siguiente:

```
/
├── lib/
├── utils/
│   ├── __init__.py
│   ├── string.py
│   └── time_helper.py
├── board.py
├── boot.py
├── config.py
├── env.py
├── hardware.py
├── main.py
├── modules.py
├── protocol.py
├── pubsub.py
└── storage.json
```

Cada uno de estos archivos y carpetas cumple un rol específico dentro de la arquitectura:

- **Carpetas `lib/` y `utils/`**
Rol: Contienen librerías de terceros (`lib/`) y utilidades propias del proyecto (`utils/`) como helpers de tiempo o formateo de texto.
- **`main.py`**
Rol: Orquestador principal.
Interacción: Es el punto de entrada de la aplicación. Inicializa los sistemas en el orden correcto (`config`, `hardware`, `modules`) y ejecuta el bucle principal.
- **`env.py`**
Rol: Configuración estática y declarativa.
Interacción: Actúa como la base para la estructura del sistema. `config.py` lo lee al arrancar para saber qué hardware, módulos y configuraciones existen.
- **`config.py`**
Rol: Gestor de configuración dinámica con persistencia.
Interacción: Carga la configuración base de `env.py` y la fusiona con los cambios guardados en `storage.json`, permitiendo que los ajustes persistan entre reinicios.
- **`hardware.py`**
Rol: Capa de Abstracción de Hardware (HAL).
Interacción: Lee su configuración desde el **ConfigManager** para inicializar los drivers de los dispositivos. En el bucle principal, lee los sensores y publica eventos de hardware a través de `pubsub.py`.
- **`modules.py`**
Rol: Lógica de la aplicación.
Interacción: Contiene las clases que definen el comportamiento del sistema. Cada módulo lee su configuración, se suscribe a eventos de hardware (o de otros módulos) y publica eventos sobre sus propios cambios de estado.
- **`pubsub.py`**
Rol: Motor de eventos (Publish/Subscribe).
Interacción: Es el que conecta a todos los componentes de forma desacoplada. Los módulos y el hardware dependen de él para comunicarse rápidamente a través de la publicación y suscripción a eventos.
- **`board.py`**
Rol: Memoria de estados.
Interacción: Es la pizarra de estados, tanto el hardware como los módulos la usan para escribir y leer estados, sirve como puente.
- **Archivos de Soporte**
 - **`boot.py`**: Se ejecuta una sola vez al arrancar el dispositivo. Usualmente se mantiene vacío para que `main.py` tome el control.
 - **`protocol.py`**: Define la estructura de paquetes para la comunicación externa.
 - **`storage.json`**: Archivo donde el **ConfigManager** guarda los valores que deben persistir.

2. CONFIGURACION DEL ENTORNO DE DESARROLLO

Para trabajar con el framework, es necesario preparar tanto la estación de trabajo (PC) como el microcontrolador (ESP32). A continuación, se detalla el proceso de instalación de herramientas y despliegue del código base.

2.1. REQUISITOS PREVIOS

HARDWARE:

- Placa de desarrollo basada en ESP32; en este caso, se utiliza el modelo **ESP32-DevKitC V4 (38 pines)**.



- Cable **USB a micro-USB**.



SOFTWARE:

- Python 3.x.
- Visual Studio Code (VS Code)
- Node.js

2.2. INSTALACION DE DRIVER

La conexión entre el ESP32 y el PC suele estar gestionada por el chip **CP2102**. Si el equipo no detecta el puerto COM correspondiente en el *Administrador de dispositivos*, es probable que sea necesario instalar los controladores para dicho chip. Por este motivo, este paso es opcional y solo se requiere si el dispositivo no es reconocido automáticamente.

El controlador puede descargarse desde la página oficial de [Silicon Labs](https://www.siliconlabs.com).

Software · 11

CP210x Universal Windows Driver	v11.4.0 12/18/2024
CP210x VCP Mac OSX Driver	v6.0.3 5/30/2025
CP210x VCP Windows	v6.7 9/3/2020
CP210x Windows Drivers	v6.7.6 9/3/2020
CP210x Windows Drivers with Serial Enumerator	v6.7.6 9/3/2020

[Show 6 more Software](#)

Una vez descargado y descomprimido el archivo, se debe ejecutar el instalador correspondiente a la arquitectura del sistema (en este caso, **x64**).

CP210xVCPInstaller_x64.exe	20/11/2025 15:19	Aplicación	1,026 KB
CP210xVCPInstaller_x86.exe	20/11/2025 15:19	Aplicación	901 KB
dpinst.xml	20/11/2025 15:19	Microsoft Edge HT...	12 KB
ReleaseNotes.txt	20/11/2025 15:19	Documento de tex...	11 KB
SLAB_License_Agreement_VCP_Windows....	20/11/2025 15:19	Documento de tex...	9 KB
slabvcp.cat	20/11/2025 15:19	Catálogo de segur...	12 KB
slabvcp.inf	20/11/2025 15:19	Información sobre...	5 KB
x64	20/11/2025 15:19	Carpeta de archivos	
x86	20/11/2025 15:19	Carpeta de archivos	

2.3. INSTALACION DE FIRMWARE DE MICROPYTHON

El framework se ejecuta sobre el intérprete de MicroPython, por lo que es fundamental instalar una versión reciente y estable en el microcontrolador.

1. Descargar Firmware:

Para obtener la versión más reciente del firmware para el módulo **ESP32-WROOM**, se debe descargar el archivo .bin desde la página oficial de [MicroPython](https://micropython.org/).

Firmware

Releases

[v1.26.1 \(2025-09-11\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\] \(latest\)](#)
[v1.26.0 \(2025-08-09\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.25.0 \(2025-04-15\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.24.1 \(2024-11-29\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.24.0 \(2024-10-25\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.23.0 \(2024-06-02\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.22.2 \(2024-02-22\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.22.1 \(2024-01-05\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.22.0 \(2023-12-27\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.21.0 \(2023-10-05\) .bin / \[.app-bin\] / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.20.0 \(2023-04-26\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.19.1 \(2022-06-18\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.18 \(2022-01-17\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.17 \(2021-09-02\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.16 \(2021-06-23\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.15 \(2021-04-18\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.14 \(2021-02-02\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.13 \(2020-09-02\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)
[v1.12 \(2019-12-20\) .bin / \[.elf\] / \[.map\] / \[Release notes\]](#)

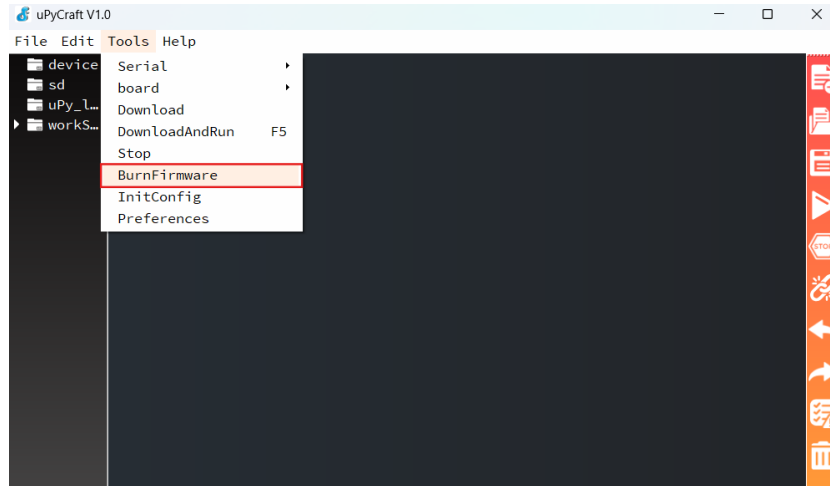
2. Instalar uPyCraft IDE:

Se procede a instalar el [IDE uPyCraft](#). Tras la descarga, se obtendrá un archivo ejecutable (por ejemplo, *uPyCraft_VX.exe*) en la carpeta de Descargas. Este entorno se utilizará para flashear el firmware en el ESP32.

3. Flasheo del Dispositivo:

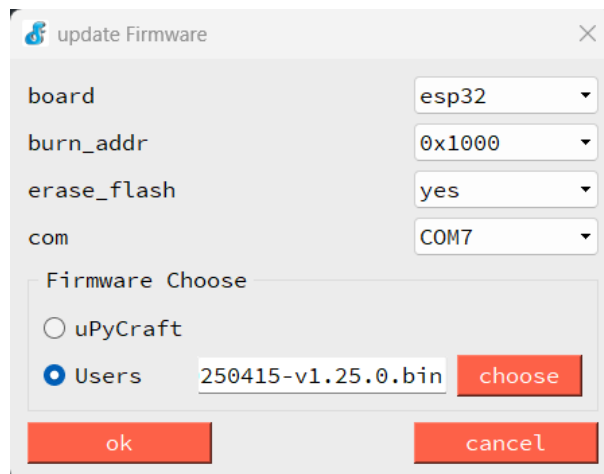
Con el ESP32 conectado al PC mediante el cable USB, identifique el puerto COM correspondiente.

Abra **uPyCraft** y seleccione la opción **Tools** → **BurnFirmware**.



En la ventana de configuración, seleccione:

- **board:** esp32
- **burn_addr:** 0x1000
- **erase_flash:** yes
- **com:** Puerto COM asignado al ESP32
- **Firmware:** Archivo .bin descargado previamente



Antes de presionar **OK**, mantenga presionado el botón **BOOT** del ESP32. Luego haga clic en **OK** y el proceso de grabación del firmware iniciará automáticamente. Una vez completado, el dispositivo quedará listo para ejecutar MicroPython.

2.4. DESPLIEGUE DEL FRAMEWORK

Una vez que el ESP32 tiene MicroPython instalado, se debe transferir la estructura de archivos del framework.

1. **Preparación en VS Code:**
 - Clone el siguiente [repositorio](#).
 - Abra la carpeta raíz del proyecto.
 - Asegúrese de que la estructura de archivos coincida con la descrita.
2. **Configuración de Pymakr:**
 - En VS Code, instale la extensión Pymakr.
 - Agregue su dispositivo. Pymakr debería detectar automáticamente el ESP32 conectado.
 - Conecte el dispositivo.
3. **Subida de Archivos:**
 - Utilice la función **"Sync project to device"** de Pymakr.
 - **Importante:** Pymakr suele ignorar ciertas carpetas por defecto. Verifique el archivo `pymakr.conf` (si existe) o la configuración de sincronización para asegurarse de que **no** se excluyan archivos críticos como `.json` si se requiere configuración inicial, o las carpetas `lib` y `utils`.
4. **Verificación del Despliegue:**

Una vez finalizada la subida, presione el botón de reinicio (EN) en el ESP32. Debería ver en la terminal los mensajes de inicialización del sistema definidos en `main.py`.

3. ARQUITECTURA DEL SISTEMA

El framework no es un simple bucle de ejecución, sino una arquitectura estratificada en **cinco capas** bien definidas. Cada capa tiene una responsabilidad única, lo que permite desacoplar la lógica de negocio de los detalles del hardware.

3.1. MODELO DE CAPAS (LAYERED ARCHITECTURE)

El sistema se organiza de la siguiente manera, desde el nivel más bajo (arranque) hasta el más alto (aplicación):

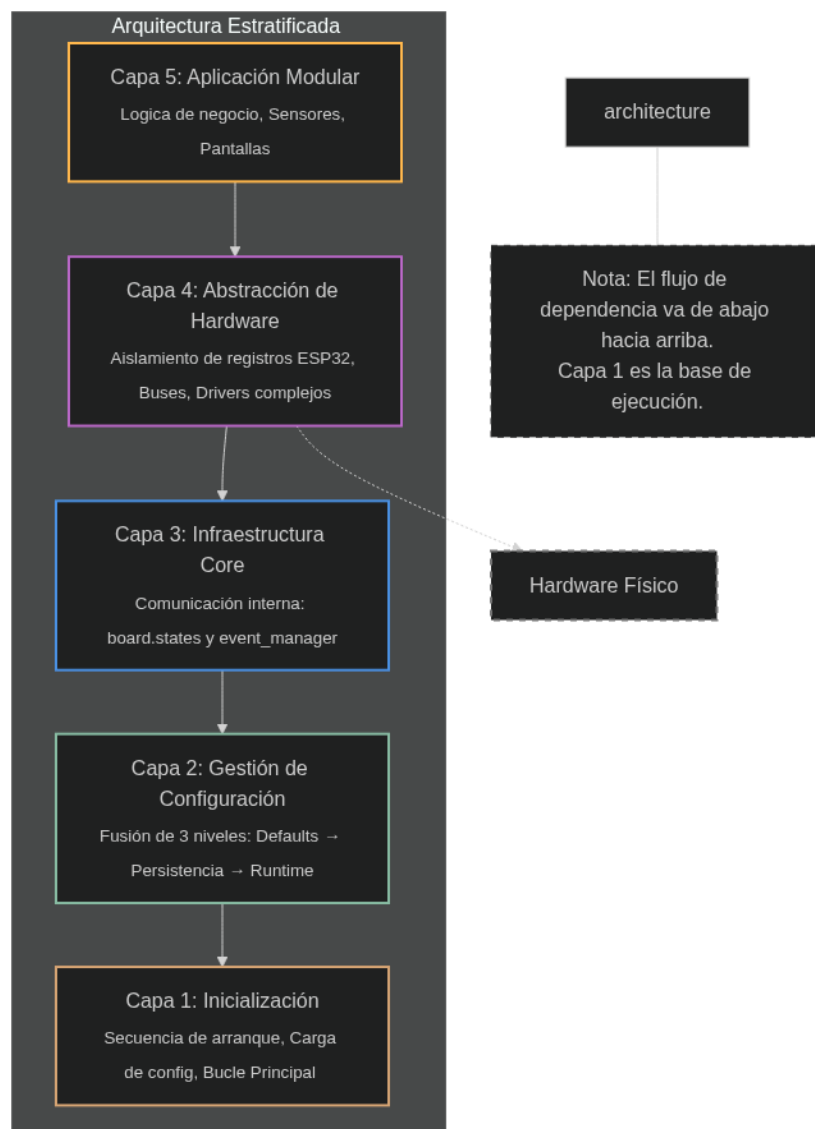
- **Capa 1: Inicialización (`main.py`, `boot.py`)**

Es el punto de entrada. Coordina la secuencia de arranque, asegura que la configuración se cargue antes que el hardware y gestiona el bucle principal (Main Loop) de 10ms.
- **Capa 2: Gestión de Configuración (`config.py`, `env.py`)**

Implementa una estrategia de fusión de tres niveles:

 1. *Defaults:* Definidos en código (`env.py`).
 2. *Persistencia:* Sobreescritos por `storage.json`.
 3. *Runtime:* Modificaciones en caliente vía red o consola.

- **Capa 3: Infraestructura Core** (`board.py`, `pubsub.py`, `protocol.py`)
Provee los mecanismos de comunicación interna. Aquí reside la "memoria" del sistema (`board.states`) y el "sistema nervioso" (`event_manager`).
- **Capa 4: Abstracción de Hardware (HAL)** (`hardware.py`)
Aísla al resto del sistema de los registros físicos del ESP32. Gestiona buses (I2C, UART), pines y drivers de dispositivos complejos.
- **Capa 5: Aplicación Modular** (`modules.py`)
Donde reside la lógica del negocio (sensores, pantallas, lógica de control). Son plugins que consumen recursos de las capas inferiores.

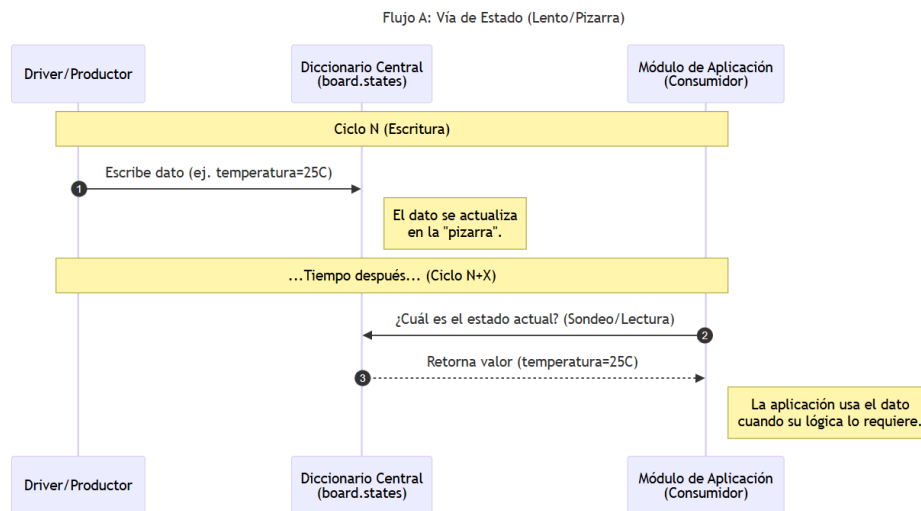


3.2. FLUJO DE DATOS: LENTO VS. REACTIVO

El framework distingue explícitamente dos caminos para el flujo de información, optimizando recursos y tiempos de respuesta:

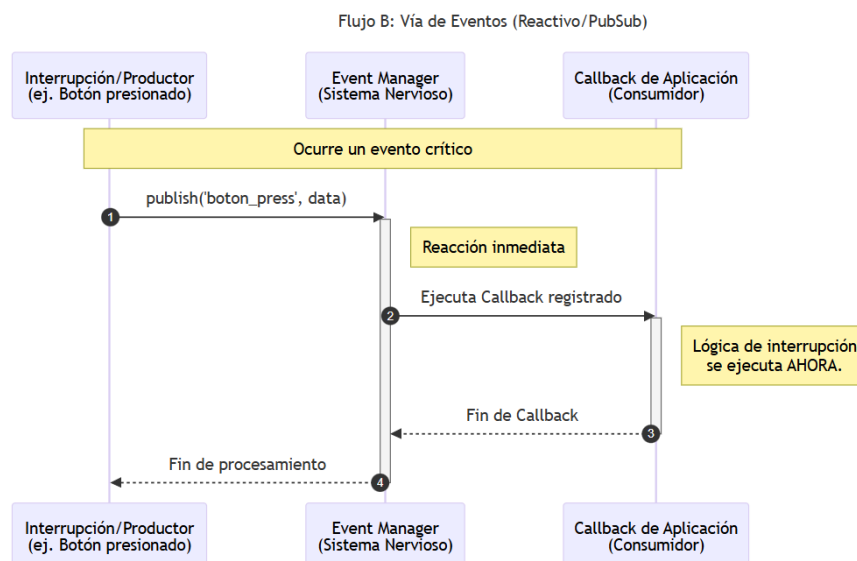
1. Vía de Estado (Board - "Memoria"):

- *Uso*: Datos de sensores, estados de relés, contadores.
- *Mecanismo*: Los drivers escriben en el diccionario board.states en cada ciclo. Los módulos leen este diccionario cuando necesitan el dato.
- *Analogía*: Consultar una pizarra de notas. Es rápido y desacoplado, ideal para procesos que no requieren inmediatez absoluta.



2. Vía de Eventos (PubSub - "Reflejos"):

- *Uso*: Interrupciones de botones, alarmas críticas, llegada de paquetes de red.
- *Mecanismo*: El productor publica `event_manager.publish('topic', data)`. El consumidor ejecuta una función *callback* inmediatamente.
- *Analogía*: Un reflejo nervioso. Se usa para reacciones instantáneas que interrumpen el flujo normal.



4. CAPA DE HARDWARE Y DRIVERS (HAL)

Esta capa es crítica para la portabilidad del código. Su objetivo es que un Módulo (Capa 5) nunca tenga que importar `machine.Pin` o saber en qué puerto GPIO está conectado un led.

4.1. NIVELES DE ENTIDADES DE HARDWARE

El archivo `hardware.py` gestiona tres tipos de entidades jerárquicas:

1. Buses (Nivel Bajo):

Son los canales de comunicación compartidos (I2C, UART). Se inicializan primero.

- *Ejemplo:* `i2c_1` en pines 21/22.

2. Drivers Genéricos (Nivel Medio):

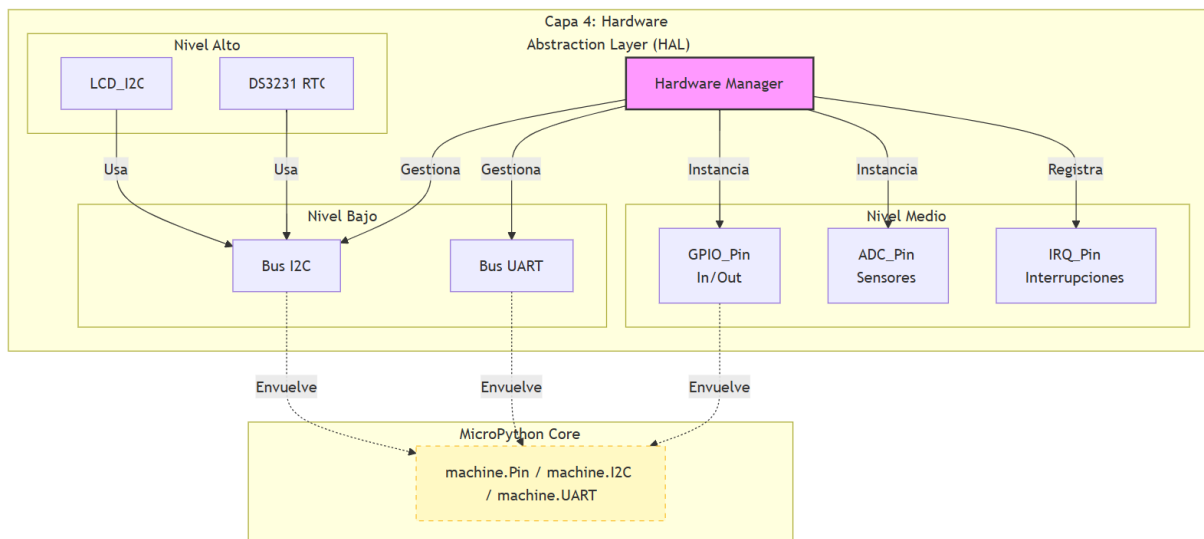
Clases que envuelven funcionalidades nativas del ESP32 para estandarizar su uso.

- *Drivers:* `GPIO_Pin` (Entradas/Salidas), `ADC_Pin` (Sensores analógicos), `IRQ_Pin` (Botones con interrupción).

3. Drivers de Dispositivo (Nivel Alto):

Controladores para hardware externo específico que utiliza los buses.

- *Ejemplo:* `DS3231` (Reloj RTC), `LCD_I2C` (Pantalla), `Keypad` (Teclado matricial).



4.2. ESTRATEGIA DE INTERRUPCIONES EN DOS ETAPAS (TWO-STAGE IRQ)

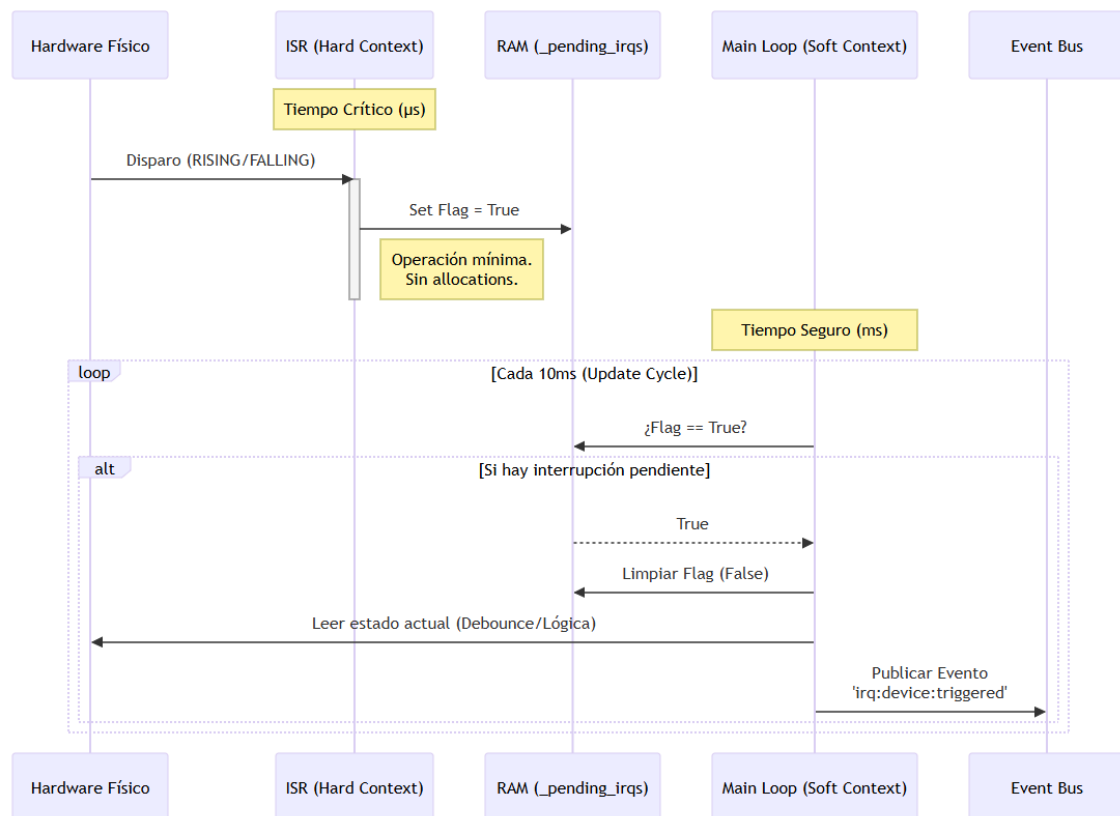
El manejo de interrupciones en MicroPython es delicado; ejecutar mucho código dentro de una interrupción real (ISR) causa reinicios y errores de memoria. Este framework soluciona esto con una estrategia de dos pasos:

1. Captura (ISR - Hard Context):

Cuando ocurre un evento físico (ej: pulsar botón), el driver solo levanta una bandera en memoria: `_pending_irqs[name] = True`. Esta operación dura microsegundos y es segura.

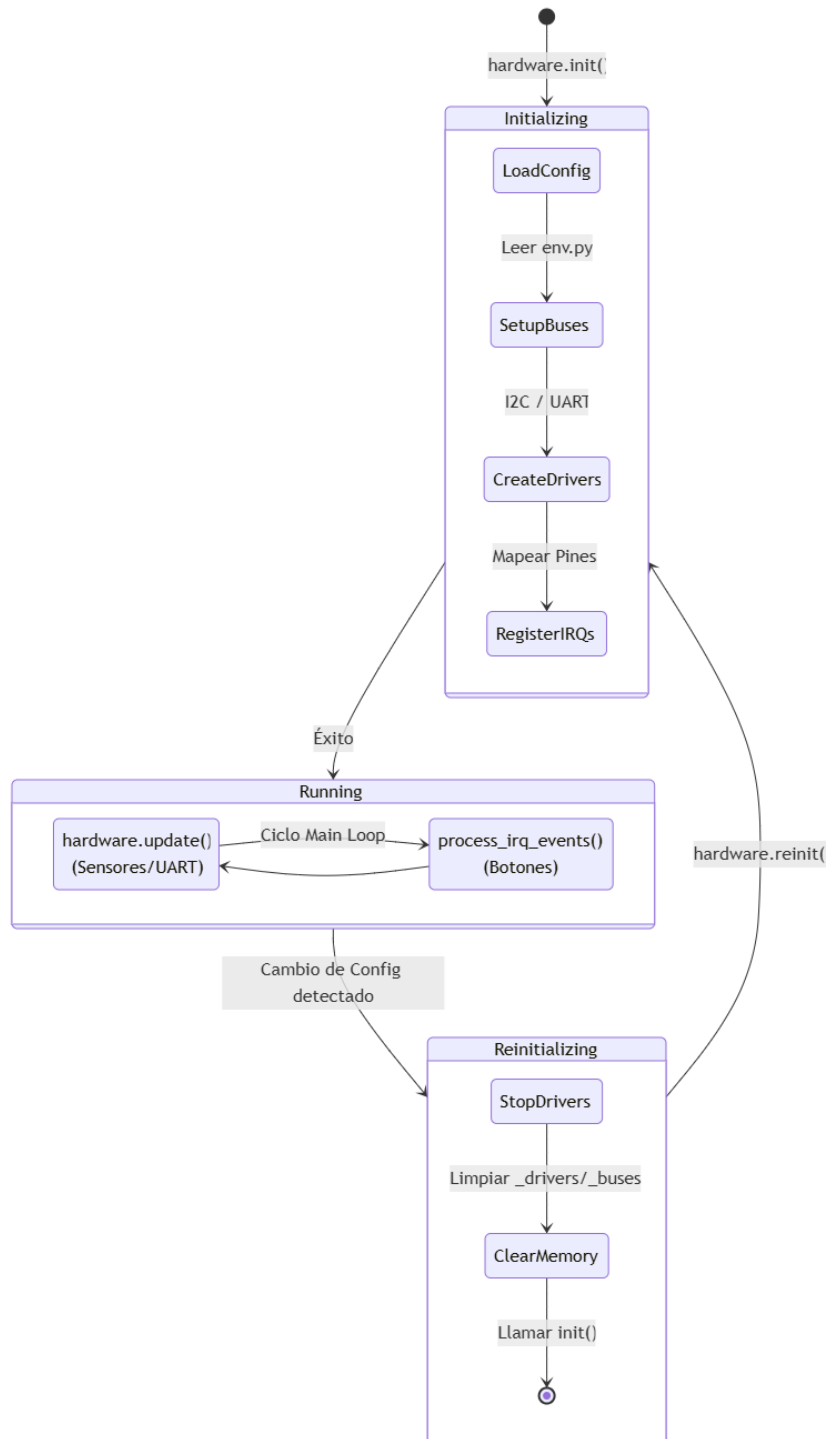
2. Procesamiento (Main Loop - Soft Context):

En el siguiente ciclo del bucle principal, la función `hardware.process_irq_events()` detecta la bandera, limpia el estado y dispara el evento PubSub correspondiente. Esto permite que la lógica de reacción (que puede ser compleja) se ejecute en un contexto seguro.



4.3. CICLO DE VIDA DEL HARDWARE

- **Init:** `hardware.init()` lee `HARDWARE_CONFIGURATION`, configura buses y luego instancia los drivers.
- **Update:** `hardware.update()` se llama cada 10ms. Aquí se hace *polling* de sensores analógicos o lectura de buffers UART.
- **Reinit:** Si la configuración cambia drásticamente, `hardware.reinit()` destruye los objetos y limpia la memoria antes de volver a inicializar, permitiendo cambios de pines sin reiniciar el ESP32.



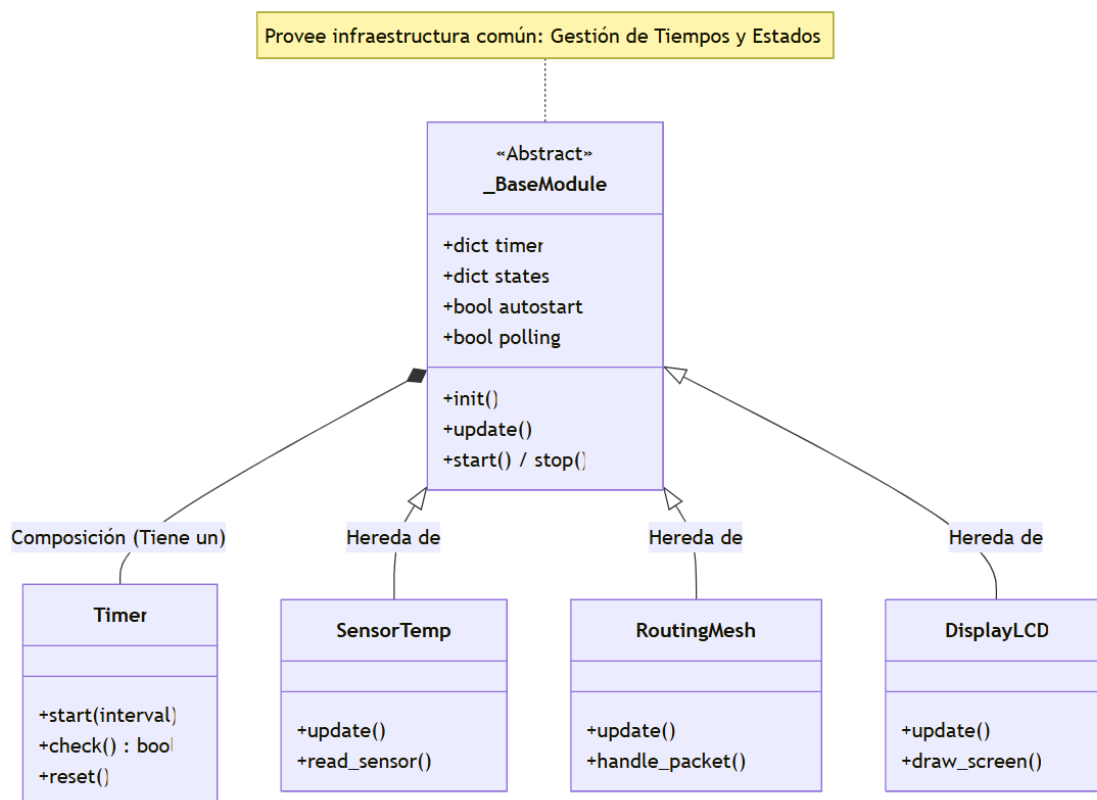
5. SISTEMA DE MODULOS (CAPA DE APLICACION)

Mientras que la capa de hardware gestiona los recursos físicos, el Sistema de Módulos (modules.py) define el comportamiento lógico del dispositivo. El framework implementa una arquitectura de **plugins**, donde cada funcionalidad (un sensor, el control de display, la lógica de red) es una clase independiente encapsulada.

5.1. LA CLASE _BASEMODULE

Todos los módulos de la aplicación heredan de la clase padre `_BaseModule`. Esta clase proporciona la infraestructura básica para que el desarrollador no tenga que reinventar la rueda en cada componente:

- **Gestión de Temporizadores:** Cada módulo tiene un diccionario interno `self.timer` con instancias de la clase `Timer`. Esto permite ejecutar tareas periódicas (ej: leer un sensor cada 10 segundos) sin bloquear el procesador usando `time.sleep()`.
- **Control de Ejecución:** Métodos estandarizados como `start()`, `stop()`, `pause()` y `resume()` permiten controlar si un módulo debe ejecutarse o quedar en espera, útil para el ahorro de energía.
- **Máquina de Estados:** Provee la estructura (`self.states`, `self.current_state`) para implementar lógica compleja de transición de estados.



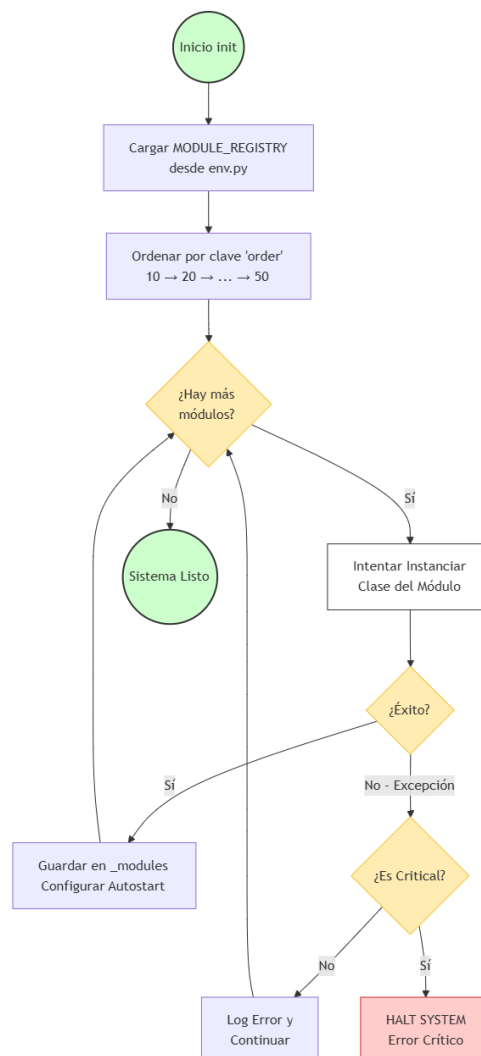
5.2. EL REGISTRO DE MÓDULOS (MODULE_REGISTRY)

El framework no carga módulos al azar. Utiliza un registro estricto definido en env.py que dicta **qué**, **cuándo** y **cómo** se carga cada componente.

```
"clock": { "class": "Clock", "order": 10, "autostart": True, "critical": True },
```

Este diccionario controla tres parámetros vitales:

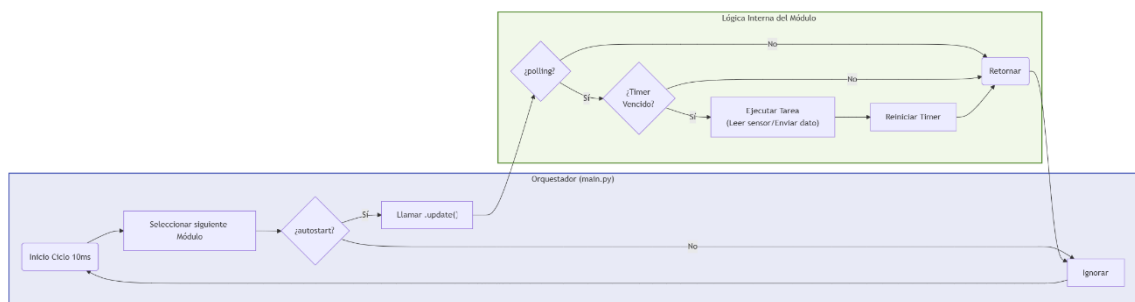
1. **order (Orden de Carga):**
Define las dependencias. Por ejemplo, el módulo Clock (orden 10) debe iniciar antes que el DataReporter (orden 50), porque este último necesita la hora correcta para enviar sus reportes.
2. **critical (Críticidad):**
Si un módulo marcado como critical falla al iniciarse (lanza una excepción), el sistema detiene el arranque para evitar un funcionamiento inestable. Si no es crítico, el error se registra y el sistema continúa con el siguiente.
3. **autostart:**
Determina si el módulo comienza a ejecutar su método update() inmediatamente tras el arranque o si espera a ser activado manualmente.



5.3. CICLO DE EJECUCIÓN DEL MÓDULO

A diferencia de los scripts lineales, los módulos funcionan bajo un esquema de **Polling Cooperativo**:

1. **Inicialización (__init__):** El módulo lee su configuración específica (MODULE_CONFIGURATION), obtiene referencias a los drivers de hardware necesarios y configura sus temporizadores iniciales.
2. **Bucle de Actualización (update):** El orquestador (main.py) llama al método .update() de cada módulo activo cada 10ms.
 - Dentro de update(), el módulo verifica sus temporizadores (self.check()).
 - Si el tiempo no se ha cumplido, retorna inmediatamente (no hace nada).
 - Si el tiempo se cumplió, ejecuta su tarea y reinicia el temporizador.
3. **Interacción:** Durante su ejecución, el módulo puede leer/escribir en board.states o publicar eventos vía pubsub.



6. PROTOCOLO Y RED (CAPA DE COMUNICACION)

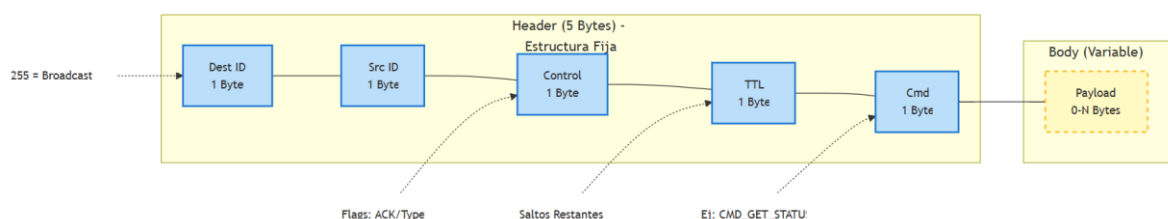
El framework incluye una pila de comunicación diseñada para redes IoT (específicamente LoRa en este caso de uso, aunque agnóstico al medio físico). Esta capa permite que el dispositivo no sea una entidad aislada, sino un nodo dentro de una red mallada (Mesh).

6.1. DEFINICIÓN DE PAQUETES (PROTOCOL.PY)

La comunicación se basa en un protocolo binario ligero para maximizar la eficiencia en medios de bajo ancho de banda. La estructura del paquete está estandarizada:

Header (5 Bytes)	Payload (Variable)
Dest ID (1B): Destinatario (255 = Broadcast)	Datos específicos del comando
Src ID (1B): Remitente	(Sensores, Configuración, Texto)
Control (1B): Flags (ACK request, Tipo de frame)	
TTL (1B): Time-To-Live (Saltos máximos en la red)	
Cmd (1B): Código de operación (ej: CMD_GET_STATUS)	

El archivo protocol.py provee las funciones build_packet() y parse_packet() para serializar y deserializar estas estructuras, abstraendo la manipulación de bytes de la lógica de los módulos.

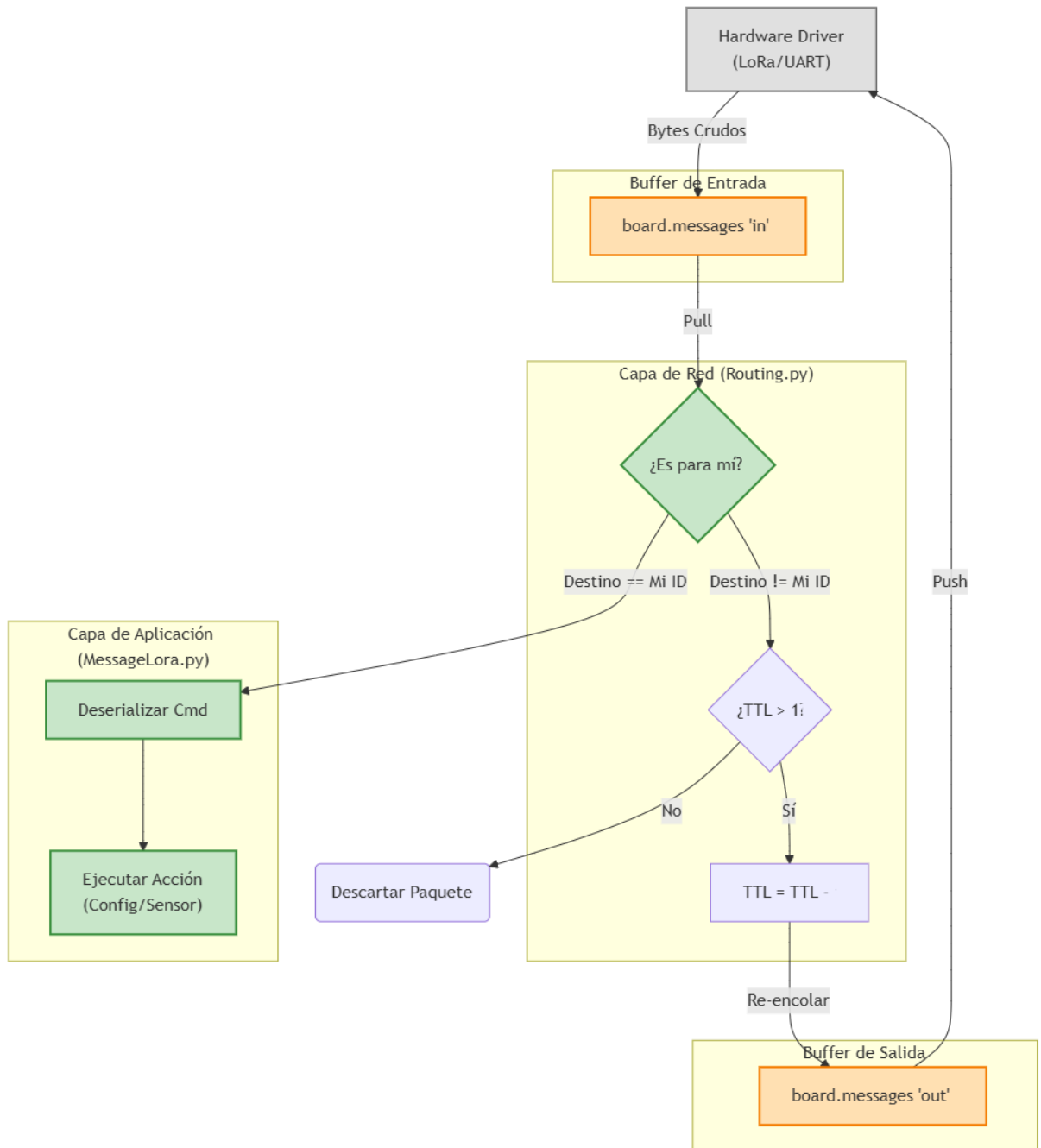


6.2. PIPELINE DE PROCESAMIENTO DE MENSAJES

El flujo de un mensaje de red atraviesa varias capas del framework:

1. **Hardware (Recepción):** El driver (ej: LoRa via UART) recibe bytes y los coloca en la cola de entrada `board.messages["uart_x"]["in"]`.
2. **Módulo de Ruteo:** Un módulo dedicado (ej: Routing) inspecciona la cola. Si el paquete no es para este nodo, decrementa el TTL y lo mueve a la cola de salida (Forwarding). Si es para este nodo, lo pasa al procesador de comandos.

3. **Módulo de Comandos:** Interpreta el byte Cmd (ej: CMD_SET_CONFIG) y ejecuta la acción localmente (ej: cambiar un valor en config_manager).



7. GUIA DE DESARROLLO

Para extender la funcionalidad del sistema, se debe seguir un flujo de trabajo estandarizado. A continuación, se detalla el proceso para crear un módulo hipotético "Blinker" (Parpadeo de LED).

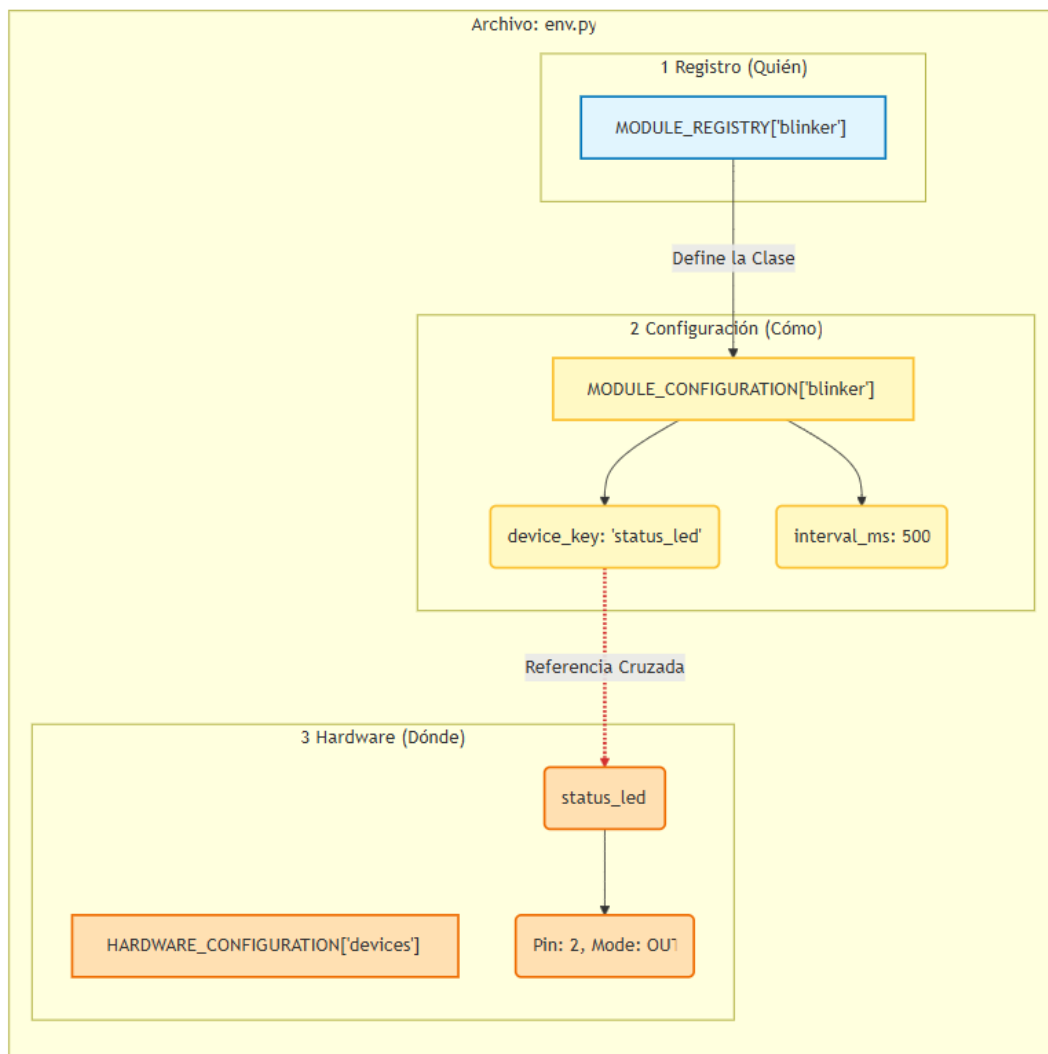
Paso 1: Declaración en env.py

Primero, definimos el hardware que usará y la configuración del módulo.

```
# En HARDWARE_CONFIGURATION['devices']:
"status_led": { "driver": "GPIO_Pin", "pin": 2, "mode": "OUT", "value": 0 },

# En MODULE_CONFIGURATION:
"blinker": { "device_key": "status_led", "interval_ms": 500 },

# En MODULE_REGISTRY:
"blinker": { "class": "BlinkerModule", "order": 99, "autostart": True, "critical":
False },
```



Paso 2: Implementación en modules.py

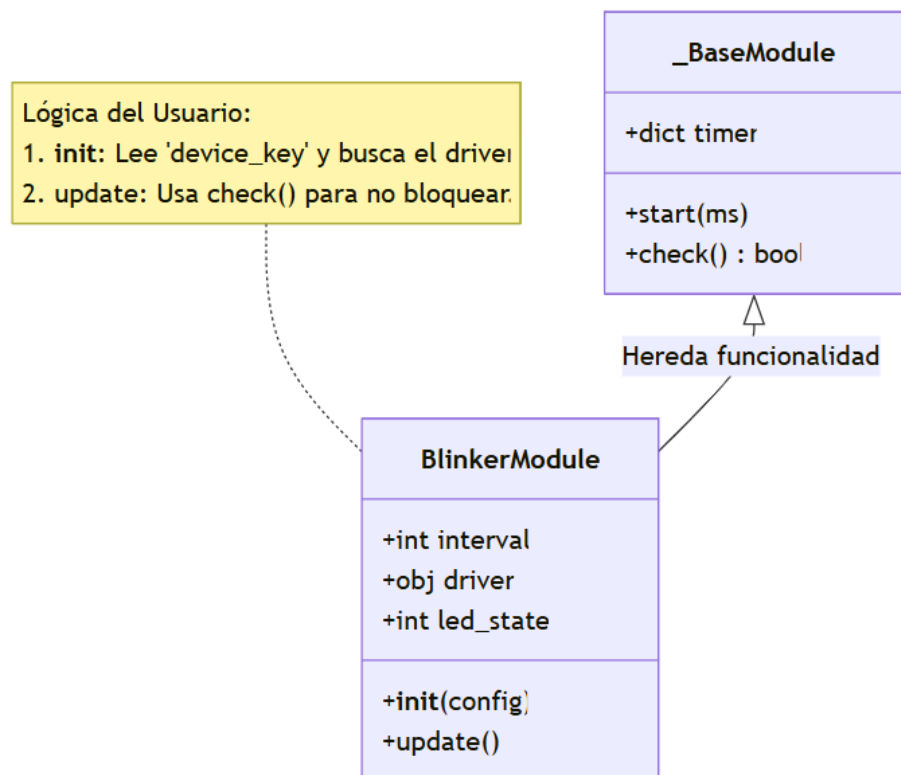
Creamos la clase heredando de `_BaseModule`.

```
class BlinkerModule(_BaseModule):
    def __init__(self, config, name=None, is_reinit=False):
        super().__init__()
        # 1. Leer configuración
        self.device_key = config.get("device_key")
        self.interval = config.get("interval_ms", 1000)

        # 2. Obtener driver de hardware
        self.driver = hardware._drivers.get(self.device_key)

        # 3. Iniciar temporizador
        self.start(self.interval)
        self.led_state = 0

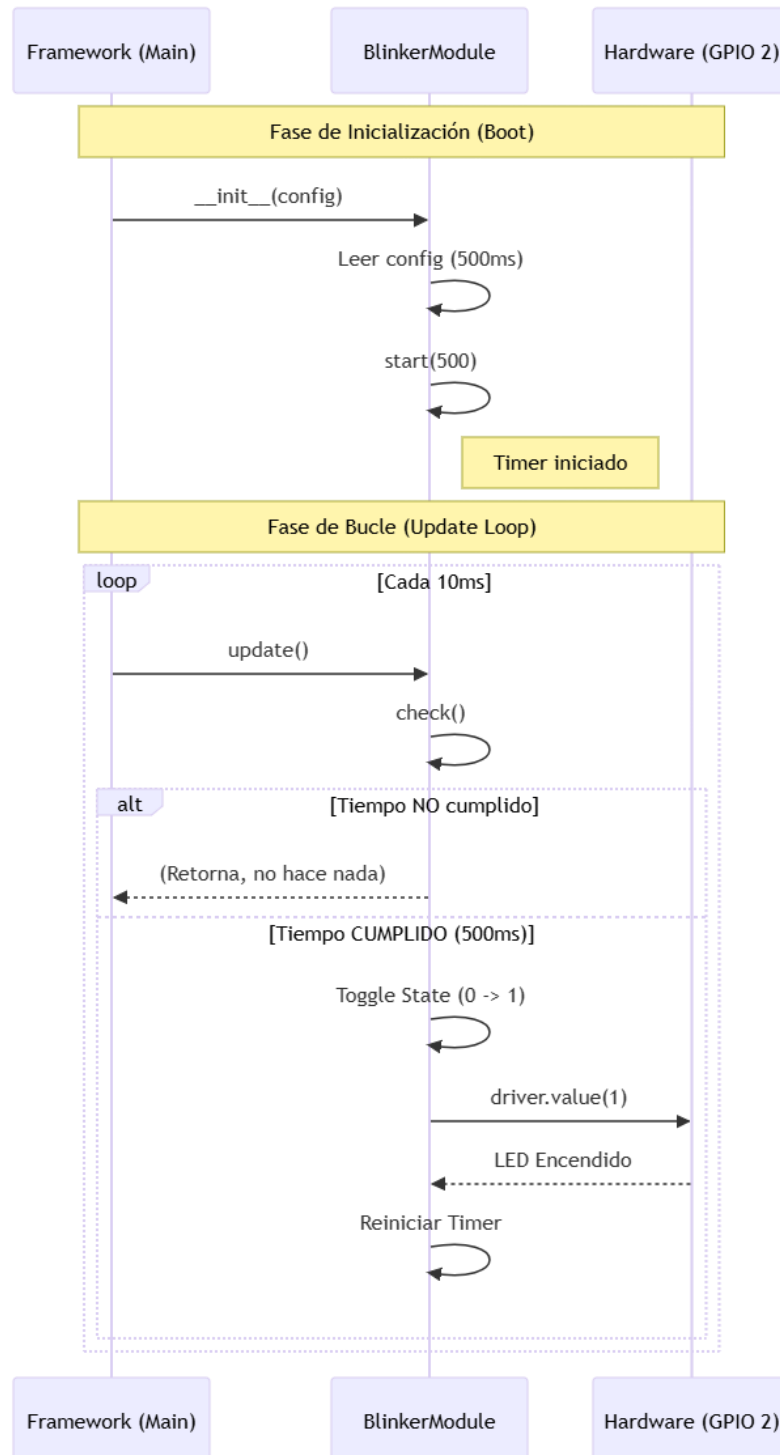
    def update(self):
        # Solo ejecuta lógica si el timer venció
        if self.check():
            if self.driver:
                self.led_state = 1 - self.led_state # Toggle
                self.driver.value(self.led_state)
                print(f"[Blinker] LED state: {self.led_state}")
```



Paso 3: Despliegue

Al subir los archivos modificados (env.py y modules.py) y reiniciar, el framework:

1. Inicializará el pin GPIO 2.
2. Instanciará BlinkerModule.
3. Ejecutará update() automáticamente, haciendo parpadear el LED cada 500ms sin bloquear el resto del sistema.



8. CONCLUSIONES

✓ **Validación de la Arquitectura Modular y Desacoplada**

El desarrollo e implementación del framework ha demostrado que la separación estricta entre la lógica de aplicación (Módulos) y el control de dispositivos físicos (HAL) es viable y eficiente en sistemas de recursos limitados como el ESP32. Esta arquitectura reduce drásticamente el acoplamiento del código, permitiendo cambiar sensores o actuadores mediante configuración (env.py) sin necesidad de reescribir la lógica de negocio, cumpliendo así con el principio de diseño Configuration-Driven.

✓ **Eficiencia del Modelo de Concurrency Híbrido**

La decisión de utilizar un bucle principal basado en polling no bloqueante con temporizadores, en lugar de asyncio, ha probado ser superior para este caso de uso específico. Este modelo garantiza una latencia determinista en la lectura de periféricos rápidos (como la recepción UART byte a byte o el barrido de teclados matriciales) mientras mantiene vivos los ciclos de vida de múltiples módulos simultáneos, evitando las condiciones de carrera comunes en sistemas multihilo.

✓ **Robustez en la Gestión de Eventos y Errores**

La implementación de la estrategia de "Interrupciones en Dos Etapas" (Two-Stage IRQ) y el sistema de mensajería PubSub ha eliminado los errores de asignación de memoria dentro de las ISR (Rutinas de Servicio de Interrupción), que son la causa más común de reinicios inesperados en MicroPython. El sistema actúa de manera resiliente: captura el evento físico instantáneamente, pero procesa la lógica pesada en un contexto seguro, garantizando la estabilidad operativa del dispositivo a largo plazo.

✓ **Escalabilidad del Ecosistema**

El sistema de registro de módulos con gestión de dependencias (MODULE_REGISTRY) permite una escalabilidad vertical del software. Es posible agregar nuevas funcionalidades complejas (como protocolos de red adicionales o algoritmos de control PID) simplemente heredando de la clase `_BaseModule` y registrándolas, sin alterar el núcleo del sistema (main.py o hardware.py). Esto facilita el mantenimiento evolutivo y la reutilización del código en futuros proyectos del LOTE V.

✓ **Optimización del Flujo de Datos**

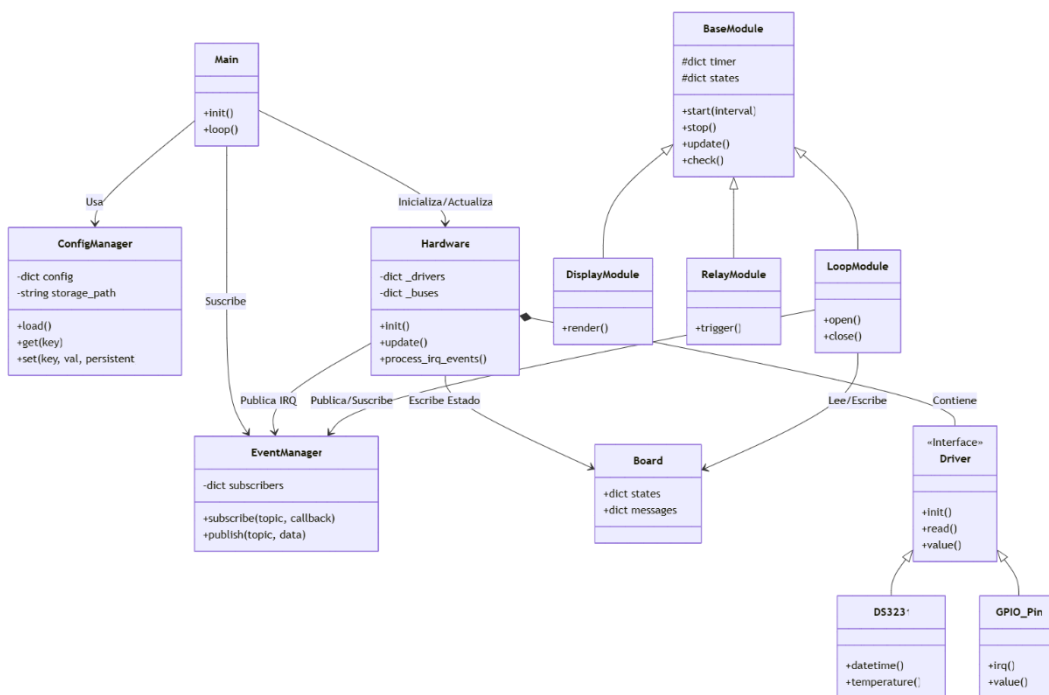
La distinción arquitectónica entre la "Memoria de Pizarra" (board.py) para datos de estado y el "Sistema Nervioso" (pubsub.py) para eventos críticos ha optimizado el uso de la CPU. Los módulos no desperdician ciclos de reloj preguntando constantemente por cambios (polling excesivo), sino que reaccionan solo cuando es necesario, resultando en un consumo energético más eficiente y una respuesta más ágil del sistema.

ANEXO A

- Enlace al repositorio:
<https://github.com/richardson2560/framework-esp32>
- Enlace a la documentación externa del repositorio:
<https://deepwiki.com/richardson2560/framework-esp32>

ANEXO B

Este diagrama muestra estáticamente cómo están construidas las clases y sus relaciones de herencia y dependencia.



ANEXO C

Este diagrama muestra dinámicamente qué pasa paso a paso cuando, por ejemplo, se presiona un botón de emergencia o sensor. Ilustra la "Interrupción en Dos Etapas".

